

Gradescope AutoGrade

User Manual

Version 0.1.0

AI-powered automated grading assistant for TAs

Repository: https://github.com/tzhazuma/gradescope_autograde

License: GPL-3.0

Platform: macOS (primary), Linux (compatible)

Python: 3.11+ (3.12 recommended)

June 2026

Contents

1	Introduction	3
1.1	Who Is This For?	3
1.2	Key Features	3
1.3	How It Works	3
2	Installation	4
2.1	Prerequisites	4
2.2	Setup Steps	4
2.3	Verify Installation	4
3	Configuration	4
3.1	Creating Your Config	4
3.2	Minimal Configuration	5
3.3	Full Configuration Reference	5
4	CLI Commands	6
4.1	login Authenticate	6
4.2	list Browse Resources	6
4.3	models Discover AI Models	6
4.4	parse-pdf Extract Questions	7
4.5	grade Run Grading Pipeline	7
4.6	review Review Flagged Grades	7
4.7	upload Submit Grades	8
4.8	export Export Results	8
4.9	tui — Interactive Terminal UI	8
4.10	gui — Web GUI	8
5	LLM Providers	9
5.1	OpenCode Go (Default, Cloud)	9
5.2	LM Studio (Local, Private)	9
6	LM Studio Auto-Management	9
6.1	Automatic Detection	10
6.2	Automatic Server Start	10
6.3	Installation Detection	10
7	Rubric Design	10
7.1	Rubric Format (YAML)	10
7.2	Best Practices	11
8	Workflow	11
8.1	Typical Grading Session	11
8.2	Resume After Interruption	12
8.3	Output Structure	12
9	Architecture	12
9.1	Design Decisions	12
10	Troubleshooting	13
10.1	Login fails	13
10.2	Model not found	13

10.3 PIP install fails on macOS	13
10.4 Grading produces unexpected results	13
10.5 Gradescope blocks requests	13
11 FAQ	13
12 License	14

1 Introduction

Gradescope AutoGrade is an AI-powered tool that automates the grading workflow on Gradescope. It fetches student submissions, evaluates them using large language models (LLMs) against configurable rubrics, and uploads scores back to Gradescope all with minimal human intervention.

1.1 Who Is This For?

This tool is designed for **Teaching Assistants (TAs)** and **instructors** who:

- Regularly grade large numbers of student submissions on Gradescope
- Want to reduce repetitive grading work
- Need consistent, rubric-based evaluation
- Prefer to keep a human in the loop for uncertain cases

1.2 Key Features

- **Automated submission fetching** downloads student answers from Gradescope via HTTP scraping
- **AI grading** evaluates answers against YAML-defined rubrics using LLMs
- **Multiple LLM backends** OpenCode Go (cloud), LM Studio (local/private), OpenAI, Anthropic
- **PDF question parsing** extracts questions and reference answers from instructor PDFs
- **Confidence-based review queue** flags uncertain grades (confidence < 70%) for human review
- **Progress tracking** resumes interrupted grading runs without re-processing
- **CSV/JSON export** exports results in Gradescope-compatible format
- **Interactive TUI & Web GUI** Terminal UI and cross-platform web interface
- **LM Studio auto-management** Auto-detect, start, and stop local models
- **10 CLI commands** full workflow via terminal

1.3 How It Works

Info

Important: Gradescope does **not** provide a public API. All interaction is performed by scraping the same HTML pages you see in your browser, using your instructor/TA credentials.

The tool operates in four layers:

1. **Transport Layer** Manages HTTP sessions, CSRF authentication, cookie persistence, and rate limiting.

2. **Client Layer** Scrapes Gradescope HTML pages to list courses, assignments, and submissions, and to upload grades.
3. **Grading Engine** Sends rubrics and student answers to an LLM, parses the structured JSON response.
4. **Workflow Orchestrator** Coordinates the full pipeline: fetch → grade → review → export → upload.

2 Installation

2.1 Prerequisites

- **Python 3.11+** (Python 3.12 is recommended)
- **macOS** (primary platform; Linux is largely compatible)
- **Gradescope account** with instructor or TA access

2.2 Setup Steps

1. Clone the repository:

```
git clone https://github.com/tzhazuma/gradescope_autograde.git
cd gradescope_autograde
```

2. Create and activate a virtual environment:

```
python3.12 -m venv venv
source venv/bin/activate
```

3. Install the package:

```
pip install -e .
```

4. (Optional) Install browser automation support:

```
pip install -e ".[browser]"
playwright install chromium
```

2.3 Verify Installation

```
gs-autograde --help
```

You should see a list of 10 available commands: `export`, `grade`, `gui`, `list`, `login`, `models`, `parse-pdf`, `review`, `tui`, `upload`.

3 Configuration

3.1 Creating Your Config

Copy the example configuration and edit it:

```
cp config/config.example.yaml config/config.yaml
```

Warning

config/config.yaml is git-ignored. Never commit your credentials to version control.

3.2 Minimal Configuration

For a quick start, you only need to set your email and password:

```
auth:
  email: "your-email@university.edu"
  password: "your-password"

llm:
  provider: "opencode-go"
  model: "deepseek-v4-flash"
```

Set your API key:

```
export OPENCODE_GO_API_KEY="your-api-key"
```

3.3 Full Configuration Reference

```
# Authentication
auth:
  email: "your-email@university.edu"
  password: ""
  cookie: "" # Browser session cookie

# Gradescope Settings
gradescope:
  base_url: "https://www.gradescope.com"
  request_delay: 1.0 # Seconds between requests
  max_retries: 3

# LLM Grading Engine
llm:
  provider: "opencode-go" # opencode-go | lmstudio
  model: "deepseek-v4-flash"
  multimodal_model: "mimo-v2.5"
  api_key: "${OPENCODE_GO_API_KEY}"
  base_url: "https://opencode.ai/zen/go/v1"
  temperature: 0.1
  max_tokens: 4096

# LM Studio (Local)
lmstudio:
  base_url: "http://localhost:1234/v1"
  auto_discover: true
  default_model: ""

# Workflow
workflow:
  auto_upload: false
  review_threshold: 0.7 # Confidence < 0.7 review
  batch_size: 50

# Output
```

```
output:
  grades_dir: "data/output/grades/"
  format: "gradescope_csv"
  generate_feedback: true
```

4 CLI Commands

4.1 login Authenticate

```
# Interactive login (prompts for credentials)
gs-autograde login

# Email-based login
gs-autograde login --email user@university.edu

# Cookie-based (from browser DevTools Application Cookies)
gs-autograde login --cookie "_gradescope_session=abc123..."

# Browser-based (opens Chromium, you log in manually, cookies auto-
  captured)
gs-autograde login --browser

# Browser-based with custom timeout (default: 120s)
gs-autograde login --browser --timeout 300
```

Successful login saves cookies to `.cookies/session.txt`.

The `-browser` flag opens a Chromium window (via Playwright) where you log into Gradescope manually. After successful login, the tool automatically extracts session cookies from the browser. This is the most reliable login method as it handles any CAPTCHA or 2FA challenges that Gradescope may present. Requires Playwright: `pip install -e "[browser]"`.

4.2 list Browse Resources

```
gs-autograde list courses
gs-autograde list assignments <COURSE_ID>
gs-autograde list submissions <COURSE_ID> <ASSIGNMENT_ID>
```

Outputs results as formatted Rich tables.

4.3 models Discover AI Models

```
gs-autograde models
```

Displays a table of available models from all configured providers:

Provider	Model	Context	Multimodal	Loaded
opencode-go	DeepSeek V4 Flash	1,000,000	No	
opencode-go	MiMo V2.5	1,000,000	Yes	
lmstudio	Gemma 4 12B	131,072	Yes	
lmstudio	Qwen 3.5 9B	262,144	No	

For LM Studio, models are queried in real-time via `GET /api/v1/models`. Only downloaded models are shown, with a checkmark indicating whether they are loaded in memory.

4.4 parse-pdf Extract Questions

```
# Display extracted questions
gs-autograde parse-pdf exam_questions.pdf

# Save as rubric template
gs-autograde parse-pdf exam_questions.pdf -o rubric.yaml

# Custom question separator
gs-autograde parse-pdf exam_questions.pdf --separator "###_Problem"
```

Info

The PDF should separate questions with a consistent marker like **## Question**. The answer section begins after a blank line or an **Answer:** prefix. Edit the generated YAML to add per-question rubric criteria and point values.

4.5 grade Run Grading Pipeline

This is the core command. It fetches submissions, grades them, and optionally uploads scores.

```
# Dry run: grade locally, do NOT upload
gs-autograde grade 123456 789012 \
  --rubric rubric.yaml \
  --dry-run

# Full run with OpenCode Go
gs-autograde grade 123456 789012 \
  --rubric rubric.yaml \
  --provider opencode-go \
  --model deepseek-v4-flash

# Full run with local LM Studio model
gs-autograde grade 123456 789012 \
  --rubric rubric.yaml \
  --provider lmstudio \
  --model gemma4-12b
```

Tip

Always start with `-dry-run` to verify your rubric produces reasonable grades before uploading anything to Gradescope.

4.6 review Review Flagged Grades

```
gs-autograde review
```

Presents each flagged submission interactively:

- Student name and question
- AI-assigned score with confidence level
- AI-generated feedback
- Options: `[a]pprove`, `[r]eject` (with notes), `[s]kip`

4.7 upload Submit Grades

```
gs-autograde upload 123456 789012 --results results.json
```

4.8 export Export Results

```
gs-autograde export --format gradescope      # Gradescope bulk upload
      CSV
gs-autograde export --format detailed         # Per-criterion breakdown
gs-autograde export --format json            # Full JSON
```

4.9 tui — Interactive Terminal UI

```
gs-autograde tui
```

Launches a full-screen Textual-based terminal interface. The TUI guides you through:

1. **Course Selection** — Browse and select from your Gradescope courses. Use **Tab** to focus the list, **Space** to toggle selection, then **Tab+Enter** to continue.
2. **Assignment Selection** — Pick the assignment to grade.
3. **Configuration** — Set file paths, choose AI model, enter grading instructions.
4. **Grading Progress** — Watch live grading with a progress bar and log output.
5. **Results** — View scores, confidence levels, and flagged items in a sortable table.

Tip

Keyboard navigation: **Tab** to move focus, **Space** to toggle checkboxes, **Enter** to activate, **Escape** to go back, **q** to quit. Selected items appear in **bold**, unchecked items in dim text.

4.10 gui — Web GUI

```
gs-autograde gui                                # Default: http
      ://127.0.0.1:8080
gs-autograde gui --port 3000                    # Custom port
gs-autograde --config my-config.yaml gui        # Custom config path
```

Opens a cross-platform web interface in your browser with a 5-step grading wizard:

1. **Login** — Authenticate with your Gradescope credentials (stored in session for subsequent steps).
2. **Select Assignment** — Choose course, fetch assignments, pick one.
3. **Configure** — Upload question/answer PDFs, rubric YAML (auto-parsed), set extra instructions, pick provider (OpenCode Go / LM Studio) and model.
4. **Grade** — Run grading with live progress display and log output.
5. **Export** — Download results as Gradescope CSV, detailed CSV, or JSON.

5 LLM Providers

5.1 OpenCode Go (Default, Cloud)

```
llm:
  provider: "opencode-go"
  model: "deepseek-v4-flash"          # 1M context, fast, text-only
  multimodal_model: "mimo-v2.5"      # 1M context, text+image+video+
  audio
```

- **deepseek-v4-flash**: 284B total params (13B active), 1M token context, 384K max output. Best for text-only grading.
- **mimo-v2.5**: 310B total params (15B active), 1M token context, 128K max output. Use when student submissions include images, diagrams, or handwriting.

Set the environment variable: `export OPENCODE_GO_API_KEY="your-key"`

5.2 LM Studio (Local, Private)

```
llm:
  provider: "lmstudio"
lmstudio:
  auto_discover: true
  base_url: "http://localhost:1234/v1"
```

1. Install [LM Studio](#)
2. Download models via LM Studio's UI
3. Start the local server (Developer → Local Server → Start)
4. Run `gs-autograde models` to see available models

Recommended models for grading:

Model	Size	Context	Best For
Gemma 4 12B	12B	131K	Balanced, general grading
Qwen 3.5 9B	9B	262K	Code/math grading
Gemma 4 E4B	4.5B	131K	Quick checks, fast iteration
Qwen 3.5 4B	4B	262K	Lightweight, resource-constrained

Warning

Privacy note: When using cloud providers (OpenCode Go, OpenAI, Anthropic), student answers are sent to external servers. Check your university's data policy. LM Studio keeps all data on your machine.

6 LM Studio Auto-Management

The LM Studio provider includes automatic server lifecycle management. No need to manually start the LM Studio desktop app before grading.

6.1 Automatic Detection

```
from gradescope_autograde.lmstudio.manager import detect_lmstudio,
    get_install_instructions

status = detect_lmstudio()
# Returns: {installed: bool, lms_path: str, server_running: bool, ...}
```

6.2 Automatic Server Start

```
from gradescope_autograde.lmstudio.manager import LmsManager

with LmsManager() as lm:
    lm.ensure_running()    # Starts server if not already running
    # ... use LM Studio for grading ...
# Server stops when exiting the context manager
```

Tip

Both the TUI and GUI use `LmsManager` automatically. When you select LM Studio as your provider, the server is started for you.

6.3 Installation Detection

If LM Studio is not installed, the tool provides platform-specific instructions:

- **macOS:** Download from <https://lmstudio.ai/> or run `curl -fsSL https://lmstudio.ai/install.sh | bash`
- **Linux:** `curl -fsSL https://lmstudio.ai/install.sh | bash`
- **Windows:** Download installer from <https://lmstudio.ai/>

7 Rubric Design

Rubrics are the most critical component of the grading pipeline. The AI's accuracy depends almost entirely on how well your rubric is structured.

7.1 Rubric Format (YAML)

```
assignment: "Homework 1: Recursion"
total_points: 100

questions:
  - id: "q1"
    title: "Define recursion"
    max_points: 10
    type: short_answer
    rubric:
      - criterion: "Correct definition"
        points: 3
        description: "Must mention function calling itself"
      - criterion: "Base case mentioned"
        points: 3
        description: "Must mention stopping condition"
      - criterion: "Example provided"
```

```

        points: 2
      - criterion: "Clarity and terminology"
        points: 2

grading_guidelines:
  - "Award partial credit when student shows understanding"
  - "Blank 0 points, flag for review"
  - "Off-topic 0 points, flag for review"

```

7.2 Best Practices

1. **Be specific.** “Correct answer” is vague. “Must include both average $O(n \log n)$ and worst-case $O(n^2)$ ” is specific.
2. **Use checklist-style criteria.** Research shows checklist rubrics produce more consistent AI grades than holistic rubrics.
3. **Include point values per criterion.** The AI needs to know how to weight each aspect.
4. **Add descriptions.** The description field tells the AI *what to look for* in the student’s answer.
5. **Use extra_instructions.** For edge cases or exceptions:

```

extra_instructions: >
  Accept either  $O(n \log n)$  or  $O(n^2)$  if properly justified.
  Deduct 1 point for missing base case explanation.

```

6. **Start with a dry run.** Always test your rubric on a few submissions with `-dry-run` before running on the full class.

8 Workflow

8.1 Typical Grading Session

1. **Prepare:** Create or parse your rubric YAML.
2. **Login:** `gs-autograde login`
3. **Dry run:** `gs-autograde grade ... -dry-run`
4. **Review rubric:** Check the dry-run results. Adjust rubric if needed.
5. **Grade:** `gs-autograde grade ... -rubric rubric.yaml`
6. **Review:** `gs-autograde review approve/reject` flagged items.
7. **Export:** `gs-autograde export -format gradescope`
8. **Upload:** Either use `gs-autograde upload` or manually upload the CSV.

8.2 Resume After Interruption

If grading is interrupted (network loss, crash, Ctrl+C), simply re-run the same **grade** command. The progress tracker (`.grading_state.json`) remembers which submissions were already processed.

To force a full re-grade, delete the state file:

```
rm .grading_state.json
```

8.3 Output Structure

```
data/output/grades/
results.json           # Full results with breakdowns
gradescope_upload.csv  # For manual Gradescope upload
detailed_report.csv    # Per-criterion breakdown
review_queue.json      # Flagged submissions
```

9 Architecture

CLI (Click + Rich)

Workflow Pipeline (orchestrator)

Gradescope Client (HTTP scraping)

- Transport (CSRF auth, cookies, rate limiting)
- HTML Parser (course/assignment/submission extraction)
- Browser Fallback (Playwright cookie extraction)

Grading Engine

- LLM Providers
 - OpenCodeGoProvider (deepseek-v4-flash, mimo-v2.5)
 - LMStudioProvider (local gemma4, qwen3.5)
 - OpenAIProvider (optional)
 - AnthropicProvider (optional)
- Rubric Parser (YAML)
- PDF Parser (pymupdf)
- Review Queue (confidence threshold)

9.1 Design Decisions

- **No official API.** Gradescope does not provide a public API. All interaction uses HTTP scraping of HTML pages.
- **CSRF authentication.** The login flow scrapes Rails CSRF tokens from the login page.
- **Dict-based interfaces.** Internal modules communicate via plain Python dicts, not strongly-typed models, for loose coupling.
- **LM Studio auto-discovery.** The LM Studio provider queries `GET /api/v1/models` in real-time to show available local models.
- **Confidence-based review.** Grades with confidence below the configured threshold (default 0.7) are automatically flagged for human review.

10 Troubleshooting

10.1 Login fails

- Check your email/password in `config/config.yaml`
- Try the cookie-based login: extract `_gradescope_session` from your browser's DevTools → Application → Cookies
- Run `gs-autograde login -cookie "..."`

10.2 Model not found

- Run `gs-autograde models` to see available models
- For LM Studio: ensure the server is running on port 1234
- For OpenCode Go: verify `OPENCODE_GO_API_KEY` is set

10.3 PIP install fails on macOS

- Ensure you're using Python 3.11+ (Python 3.14 on Homebrew may have issues)
- Try: `python3.12 -m venv venv && source venv/bin/activate && pip install -e .`

10.4 Grading produces unexpected results

- Check your rubric are the criteria specific enough?
- Run with `-dry-run` and review the AI's feedback
- Lower `review_threshold` in config to catch more items for human review
- Try a different model (e.g., switch from `deepseek-v4-flash` to a larger local model)

10.5 Gradescope blocks requests

- Increase `request_delay` in config (try 2.0 or 3.0 seconds)
- Reduce `batch_size` to process fewer submissions per run
- Use the browser fallback: install with `pip install -e ".[browser]"`

11 FAQ

Does this work with any Gradescope course?

Yes, as long as you have instructor or TA access.

Is my data private?

Cloud providers (OpenCode Go, OpenAI) receive student answers. LM Studio keeps everything on your machine.

How accurate is AI grading?

Research shows LLMs match human accuracy on structured, rubric-based tasks. Always spot-check and use the review queue.

Can I grade multiple assignments at once?

Run separate terminal sessions for different assignments. Each maintains its own progress state.

What if I need to regrade?

Delete `.grading_state.json` and re-run. Or edit your rubric and re-grade only flagged submissions.

12 License

Gradescope AutoGrade is licensed under the GNU General Public License v3.0 (GPL-3.0).

Repository: https://github.com/tzhazuma/gradescope_autograde

Issues: https://github.com/tzhazuma/gradescope_autograde/issues